

Assignment 2: Building a Small-Scale Foundation Model from Scratch

Jiabei Liu

liu.jiabe@northeastern.edu

1. Introduction

Transformer architectures have revolutionized natural language processing by relying entirely on attention mechanisms rather than recurrent or convolutional structures. GPT (Generative Pretrained Transformer) models, in particular, employ a stack of transformer **decoder blocks** to model language in an autoregressive fashion — predicting the next token given the previous ones.

The objective of this project was to **implement and train a simplified GPT-like model (Mini-GPT)** from scratch. The focus was on understanding the key architectural components of transformers—multi-head attention, positional embeddings, and feed-forward layers—while experimenting with limited computational and data resources.

2. Model Architecture

The Mini-GPT model consists of a **transformer decoder** with two layers, each built using **multi-head self-attention** and **position-wise feed-forward networks**.

Key architectural specifications:

- **Number of layers:** 2
- **Embedding dimension (d_{model}):** 128
- **Number of attention heads:** 4
- **Feed-forward dimension:** 512
- **Total parameters:** approximately **13.3 million**

Each input token is first mapped to an embedding vector, combined with learned positional encodings, and then passed through the stacked decoder blocks. The final linear layer projects the outputs back into the token vocabulary space for prediction.

(If included, the architecture diagram would show the data flow: token embeddings → positional encoding → stacked decoder blocks → linear projection → softmax output.)

3. Dataset

For training, a **tokenized text corpus** was used, tailored for small-scale experimentation. The dataset was preprocessed and truncated to a limited vocabulary and sequence length to ensure manageable training times.

- **Vocabulary size:** 1,000 tokens (reduced from GPT's 50,257)
- **Training tokens:** 90,000
- **Validation tokens:** 10,000
- **Sequence length:** 32

The reduced vocabulary and short sequences were chosen to maintain computational feasibility, though they limited the expressive capacity of the model.

4. Training Setup

The model was trained using the **Adam optimizer** with a **cross-entropy loss** function, a standard choice for language modeling tasks. Training was executed on a **GPU (NVIDIA L4)**, allowing efficient batch processing and backpropagation.

Each training iteration included:

1. **Forward pass** – computing token predictions through the model,
2. **Loss computation** – calculating cross-entropy between predictions and ground truth,
3. **Backward pass** – applying gradient updates using Adam optimizer.

The overall training time remained reasonable due to the reduced model size and dataset.

5. Hyperparameter Experiments

To understand the sensitivity of model performance to key hyperparameters, several configurations were tested. The table below summarizes the main experiments:

Config	Learning Rate	Batch Size	d_model	Layers	Final Val Perplexity
1	1e-3	64	128	2	42.5
2	5e-4	32	64	1	58.7

The results show that the larger configuration (Config 1) with a higher learning rate and embedding size achieved **lower validation perplexity**, suggesting better language modeling capacity. However, further scaling was constrained by data and compute limits.

6. Results and Observations

Training loss consistently **decreased** across epochs, indicating that the model successfully learned to fit the training data. However, the **validation loss and perplexity** began to increase after several epochs, suggesting **overfitting**.

The model effectively **memorized the training sequences** but showed limited ability to generalize to unseen text. Generated samples appeared coherent over short spans but often degenerated into repetitive or nonsensical text, a typical sign of overfitting in small-scale transformer training.

7. Challenges

Several limitations affected model performance:

- The **dataset size** was too small for stable language modeling, leading to overfitting and high validation perplexity.
 - The **restricted vocabulary** (1,000 tokens) reduced the model's expressive range.
 - More extensive datasets ($\geq 500k$ tokens) and longer training runs would likely produce more generalizable representations.
 - Limited computational budget constrained exploration of deeper models or longer sequence lengths.
-

Conclusion

The Mini-GPT implementation was **technically successful**, reproducing the core architecture and training dynamics of the GPT model. The results highlight that **model design alone is insufficient** without a sufficiently large and diverse dataset. Future work will focus on scaling the dataset, introducing regularization (e.g., dropout, weight decay), and experimenting with improved tokenization to enhance generalization.

In summary, this project demonstrates a clear understanding of the transformer architecture and provides a strong foundation for scaling toward larger, more capable GPT-style models.